SOFT COMPUTING · PROJECT REPORT

Ransomware Detection using Multi-Objective Feature Selection

Can ransomware be caught before it runs, and what does that cost in accuracy?

Algorithm: SPEA2 **Datasets:** MLRan · RD2024 · UGRansome **Status:** Complete (UGRansome: see 7.4)

OBJECTIVES

1. **To minimise the time required to detect ransomware, ideally before it executes**
2. **To maximise recall value**

As in our phishing study, the false-alarm rate is optimised alongside these two. It keeps objective 2 well-posed. See Section 4.

1. Introduction

Ransomware is malicious software that encrypts a victim's files and demands payment for the key. Hospitals, councils and companies have been shut down for weeks by a single infection.

What makes ransomware different from most malware is that **the damage is done the moment it runs.** Encryption starts within seconds; by the time a detector has watched the program misbehave, files are already lost. A security tool can judge a program in two ways:

- STATIC **read the file without running it**: its header, its embedded text strings. This takes milliseconds, and it happens *before* the program can do any harm.
- DYNAMIC **run it inside a sandbox** (an isolated test machine) and record what it does: files touched, registry keys changed, system calls made. This sees the real behaviour but takes about two minutes per file.

Dynamic features are usually more informative, so speed and accuracy pull against each other. This project applies the method from our phishing study, **SPEA2 multi-objective feature selection**, to three ransomware datasets. The goal is to find which features are worth collecting, and in particular whether a detector that never runs the file can still catch ransomware reliably.

2. Datasets

We use three public datasets chosen to differ in what they measure and in size (13 to 483 features). Only the processed feature tables are used; no malware is ever downloaded.

TABLE 1 — THE THREE DATASETS

Dataset	Source	Samples used	Features	Task
MLRan	Onwuegbuche et al., JNCA 2026; Zenodo 23403681 (CC-BY-4.0)	4,880	483	Ransomware (2,330) vs goodware (2,550)
RD2024	Hussain, *Ransomware Dataset 2024*; Zenodo 13890887 (CC-BY-4.0)	15,638	148	12 ransomware families (4,762) vs benign (10,876)
UGRansome	Nkongolo et al., *Information* 12(10), 2021; Kaggle	≈ 149k	≈ 13	Zero-day (anomaly) vs known-signature network traffic

2.1 What the features measure, and what they cost

TABLE 2 — FEATURE TIERS AND THEIR COST

Dataset	STATIC ≈ 50 ms, before execution	DYNAMIC ≈ 120 s, sandbox run
MLRan	223 text strings found inside the file (e.g. `getlogicaldrives`, `.aspack`)	260 behaviours: Windows API calls, registry and file operations, loaded DLLs, sandbox signatures
RD2024	135 PE-header fields: image base, section sizes, flags, machine type	13 behaviour counts: registry reads/writes, processes, files created, API calls
UGRansome	All features come from network flows (ports, protocol, bytes, address, Bitcoin activity). There is only one tier, so cost is measured as the number of features.	

The dynamic latency is the default analysis time of the Cuckoo sandbox (120 s). The static figure is a generous allowance for parsing a header or extracting strings. **The exact numbers do not matter**: with only two tiers, any static cost lower than any dynamic cost gives exactly the same set of best trade-offs, so no sensitivity analysis is needed.

2.2 Auditing the data before using it

> **TWO PROBLEMS FOUND, AND FIXED, BEFORE ANY OPTIMISATION**
>
> **RD2024: the sandbox's own verdict was in the data.** The column `processes_malicious` alone separates ransomware from benign with AUC 0.996. It is not a behaviour but the sandbox's *judgement*, so using it would let the optimiser copy an existing detector instead of learning one. We removed it together with the four other verdict columns (`*_suspicious`, `files_malicious`, `network_threats`). After this, no single feature exceeds AUC 0.95.
>
> **MLRan: a third of the test set also appears in training.** 320 of 975 test vectors are exact copies of training vectors. We keep the authors' split, because their 483 features were chosen on it, but **every MLRan result is also reported on the 655 unseen test rows**, which is the honest measure of generalisation.

2.3 Why these three datasets

- **MLRan and UGRansome** were specified for the project. MLRan has the largest search space we know of in a public ransomware set (483 features). UGRansome is small enough to *check every possible subset*, which gives the true optimum to measure SPEA2 against.
- **RD2024** was our choice for the third. It is recent (2024), openly licensed, covers 12 well-known families (e.g. WannaCry, LockBit, Ryuk, REvil), and has a natural

static/dynamic split. Its static features (header fields) are a different *kind* from MLRan's (text strings), so the method is tested on two independent feature types.

- Together they span **13, 148 and 483 features**. This matters: the $2^{483} \approx 10^{145}$ subsets of MLRan cannot be searched by anything simple.

3. Aim

> **THE RESEARCH QUESTION**
>
> **Can ransomware be detected from the file alone, before it runs? If so, from how few features, and what does skipping the sandbox cost in missed ransomware and false alarms?**

As before, **the classifier is the instrument, not the subject**: a Random Forest is held fixed and we study which inputs it needs. The output is not one detector but the full set of best trade-offs, from which a deployment chooses, for example a pre-execution scanner or a full sandbox pipeline.

4. The Objectives in Measurable Form

TABLE 3 — THE OBJECTIVES AS OPTIMISED (ALL MINIMISED)

Objective	Definition	Why it matters	Target
f_1 — Detection cost Objective 1	Sum of the *distinct* tiers the selected features need: 50 ms, or 120,050 ms if any dynamic feature is used. UGRansome: feature count	A static-only detector can block the file *before* it executes; one dynamic feature forces a two-minute sandbox run	Static only, if accuracy allows
f_2 — Missed ransomware Objective 2	`FNR = FN/(FN+TP)` = 1 – recall	A single missed sample can encrypt an organisation's files	Recall ≥ 95%
f_3 — False alarms supporting	`FPR = FP/(FP+TN)`	Quarantining legitimate software breaks users' work; they then disable the tool	FPR ≤ 2%

Targets are the same as in the phishing study, so the two projects are directly comparable. A tiny per-feature term (10^{-5} ms) is added to f_1 so that, between two equally good detectors in the same tier, the smaller one is preferred.

> **WHY F_3 IS ESSENTIAL, AND HOW WE SAW IT HAPPEN**
>
> Without f_3 the optimiser can label *every* file as ransomware: recall becomes 100% and the detector is useless. Such "flag everything" detectors really do appear on our Pareto fronts, because nothing beats their recall. Section 6.3 explains how we choose deployment points so that they are never selected.

5. Algorithm — SPEA2

5.1 How it works (recap)

SPEA2 (Zitzler, Laumanns and Thiele, 2001) is a multi-objective evolutionary algorithm. Each candidate is a string of yes/no bits, one per feature (483 bits for MLRan). A population of 100 candidates evolves for 50 generations by two-point crossover and bit-flip mutation; selection works as follows:

1. **Dominance.** A dominates B if A is no worse on all three objectives and better on at least one.
2. **Strength and raw fitness.** Each candidate's strength is how many it dominates; its raw fitness is the total strength of those dominating it. Zero means nothing beats it.
3. **Density.** Ties are broken by distance to the k-th nearest neighbour, so sparse regions of the trade-off are preferred and the front stays spread out.
4. **Archive.** A fixed-size archive keeps the best candidates; when it overflows, the most crowded are removed.

The output is a **Pareto front**: detectors that cannot be improved on one objective without becoming worse on another.

5.2 Why SPEA2

1. **Duplicate-heavy fitness.** Many different feature subsets give identical scores (MLRan alone has 1,274 duplicate rows). SPEA2 gives identical candidates identical fitness, and its archive truncation handles them consistently.
2. **Its overhead is invisible here.** Each evaluation trains a classifier (hundreds of milliseconds); SPEA2's extra bookkeeping costs microseconds.
3. **Weighted sums cannot do the job.** They need arbitrary weights, return one point per run, and cannot reach non-convex parts of the front. We run a weighted-sum PSO to show this, and **NSGA-II** as the standard multi-objective comparison.

5.3 What we had to change for 483 features: the starting population

The standard binary start switches each feature on with probability ½, so every initial candidate holds about 240 of MLRan's 483 features, about 130 of them dynamic. To reach a static-only detector the search must switch off *all* 260 dynamic features, but bit-flip mutation changes about one bit per child. **In practice, a static-only detector is unreachable.**

We therefore use a **sparse initial population**, the usual remedy in large-scale feature selection (cf. SparseEA). Each starting candidate gets its own density, drawn log-uniformly between 1/n (about one feature) and ½. It uses **no knowledge of which features are static**, and SPEA2, NSGA-II and PSO all receive exactly the same start.

TABLE 4 — EFFECT OF THE STARTING POPULATION ON MLRAN (SAME SETTINGS OTHERWISE)

Start	Runs	Hypervolume	Features in final front	Static-only detectors found
Standard (density ½)	6	0.121 (every run)	124–176	0 in every run
Sparse (ours)	20	1.252–1.292	1–225	47–91 per run

The standard-start runs (3 SPEA2 + 3 NSGA-II) are kept as an ablation. Their hypervolume is identical because every solution stays in the sandbox tier.

This also explains a result from our phishing study, where no-lookup detectors appeared in only some runs: at 111 features the effect was mild; at 483 it is total.

6. Methodology

1. **Load and audit.** Encode each dataset (hex fields parsed, flag lists one-hot); check balance, duplicates, train/test overlap and single-feature "shortcuts" (Section 2.2).

2. **Split.** MLRan: the authors' split (3,905 / 975). RD2024 and UGRansome: stratified 80/20. The test set is then sealed.
3. **Internal validation split.** The training data is split again 75/25. The search trains on the first part and scores on the second; it never sees the test set.
4. **Baselines.** Random Forest on all features, on static features only, and PCA (95% variance) and a mutual-information filter (top 25).
5. **Search.** SPEA2 and NSGA-II, population 100, 50 generations, **10 seeds each** per dataset; weighted-sum PSO with four weightings.
6. **Choose deployment points** from the pooled Pareto front, on validation data (6.3).
7. **Final validation.** Retrain a 200-tree Random Forest on each chosen subset and evaluate once on the sealed test set (MLRan also on its 655 unseen rows).

6.1 How one candidate is scored

```
if no features selected  →  worst value on every objective

f₁ = 50 ms (+120,000 ms if any dynamic feature) + 10⁻⁵ × n_features
     train a 30-tree Random Forest on the selected features
f₂ = FN / (FN + TP)      ┐ from one confusion matrix
f₃ = FP / (FP + TN)      ┘ on the validation split
```

6.2 Measuring search quality

Hypervolume (reference point 1.1, 1.1, 1.1 on normalised objectives; maximum 1.331) measures how much of the objective space a front covers. SPEA2 and NSGA-II are compared over 10 seeds with a two-sided **Mann–Whitney U test**. Wall-clock time was unreliable (the laptop slept during some runs), so search effort is reported as **classifier evaluations**.

6.3 Choosing a deployment point from the front

For each cost tier we pick the Pareto solution with the **lowest balanced error, (FNR + FPR) / 2**, on validation data. This gives a *pre-execution* detector (best static-only) and a *full-analysis* detector (best overall).

> **A CORRECTION WE MADE, REPORTED OPENLY**
> Our first rule, copied from the phishing study, was "highest recall in the tier". Here it picked "flag everything" detectors (e.g. 94.6% recall at **64% false alarms**), because those are Pareto-optimal on recall. Balanced error needs no hand-picked target and penalises both mistakes equally. The rule uses validation data only; the test set played no part in choosing it.

Tools: `Python 3` · `pymoo` (SPEA2, NSGA-II, hypervolume) · `pyswarms` (binary PSO) · `scikit-learn` · `scipy` · `pandas`, `matplotlib`. Everything runs on an ordinary laptop.

Datasets. F. C. Onwuegbuche et al., MLRan, Zenodo 23403681. · M. Hussain, Ransomware Dataset 2024, Zenodo 13890887. · M. Nkongolo, J. P. van Deventer, S. M. Kasongo, *UGRansome1819: A Novel Dataset for Anomaly Detection and Zero-Day Threats*, Information 12(10):405, 2021.

Algorithm. E. Zitzler, M. Laumanns, L. Thiele. *SPEA2: Improving the Strength Pareto Evolutionary Algorithm.* TIK Report 103, ETH Zurich, 2001.

7. Comparison of Algorithms

Every row retrains the same 200-tree Random Forest on the method's features and is measured **once on the sealed test set.** Cost is per file: 50 ms means no sandbox.

TABLE 5 — RD2024 (TEST SET: 3,128 PROGRAMS)

Method	Features	Cost	Accuracy	Recall	FPR	Notes
PRE-EXECUTION (NO SANDBOX)						
SPEA2 (pre-execution)	**7**	**50 ms**	97.89%	**96.12%**	**1.33%**	✓both targets, 7 header fields
NSGA-II (pre-execution)	14	50 ms	97.51%	95.80%	1.75%	✓both targets
All static features	135	50 ms	98.18%	95.91%	0.83%	✓both targets, 19× more features
PSO weighted (balanced)	68	50 ms	98.27%	96.54%	0.97%	✓both targets, 10× SPEA2's features
FULL ANALYSIS (SANDBOX)						
All features (baseline)	148	120 s	99.17%	98.74%	0.64%	Reference point
SPEA2 (full analysis)	**47**	120 s	99.17%	98.43%	**0.51%**	Same accuracy, ⅓ of the features
NSGA-II (full analysis)	70	120 s	99.30%	98.64%	0.41%	Comparison
PSO weighted (any weights)	—	—	—	—	—	No weighting reached this tier
MI filter (top 25)	25	120 s	99.10%	98.43%	0.60%	Strong, but 8 of its 25 need the sandbox
PCA (95% var.)	148 in	120 s	98.24%	95.91%	0.74%	Components need every input

Only 27 RD2024 test rows appear in training; unseen-row results differ by ≤ 0.05 points.

TABLE 6 — MLRAN (TEST SET: 975 PROGRAMS; "UNSEEN" = THE 655 NOT DUPLICATED IN TRAINING)

Method	Features	Cost	Recall	FPR	Unseen recall	Unseen FPR	Notes
PRE-EXECUTION (NO SANDBOX)							
SPEA2 (pre-execution)	**73**	**50 ms**	**93.33%**	4.90%	**94.85%**	8.74%	Best static detector
NSGA-II (pre-execution)	89	50 ms	91.18%	5.29%	91.60%	9.44%	Comparison
All static features	223	50 ms	90.97%	4.71%	91.60%	8.39%	No selection
PSO weighted (balanced)	3	50 ms	96.77%	94.71%	96.21%	91.26%	Flags almost everything
FULL ANALYSIS (SANDBOX)							
All features (baseline)	483	120 s	96.99%	3.33%	96.21%	5.94%	Reference point
SPEA2 (full analysis)	79	120 s	95.48%	5.10%	94.31%	9.09%	Over-fitted validation (7.3)
NSGA-II (full analysis)	**158**	120 s	**98.06%**	**3.33%**	**97.56%**	**5.94%**	Beats baseline, ⅓ features
PSO weighted (recall)	240	120 s	96.34%	3.53%	95.39%	6.29%	1 point per run
MI filter (top 25)	25	120 s	92.69%	13.33%	90.79%	15.03%	All 25 are sandbox features
PCA (95% var.)	483 in	120 s	98.49%	3.33%	98.10%	5.94%	Needs every input

TABLE 7 — SEARCH QUALITY ACROSS 10 INDEPENDENT RUNS EACH

Dataset	Algorithm	Median HV	Mean ± std	Worst run	Pooled front	Static-only	Evaluations / run
RD2024	SPEA2	1.298	1.298 ± 0.009	1.282	225	158	4,287
	NSGA-II	1.301	1.298 ± 0.008	1.281	203	144	4,297
MLRan	SPEA2	1.283	1.280 ± 0.010	1.257	265	170	4,572
	NSGA-II	1.280	1.276 ± 0.011	1.252	214	177	3,896

Mann–Whitney U, SPEA2 vs NSGA-II hypervolume: RD2024 p = 1.00; MLRan p = 0.68. No significant difference on either.

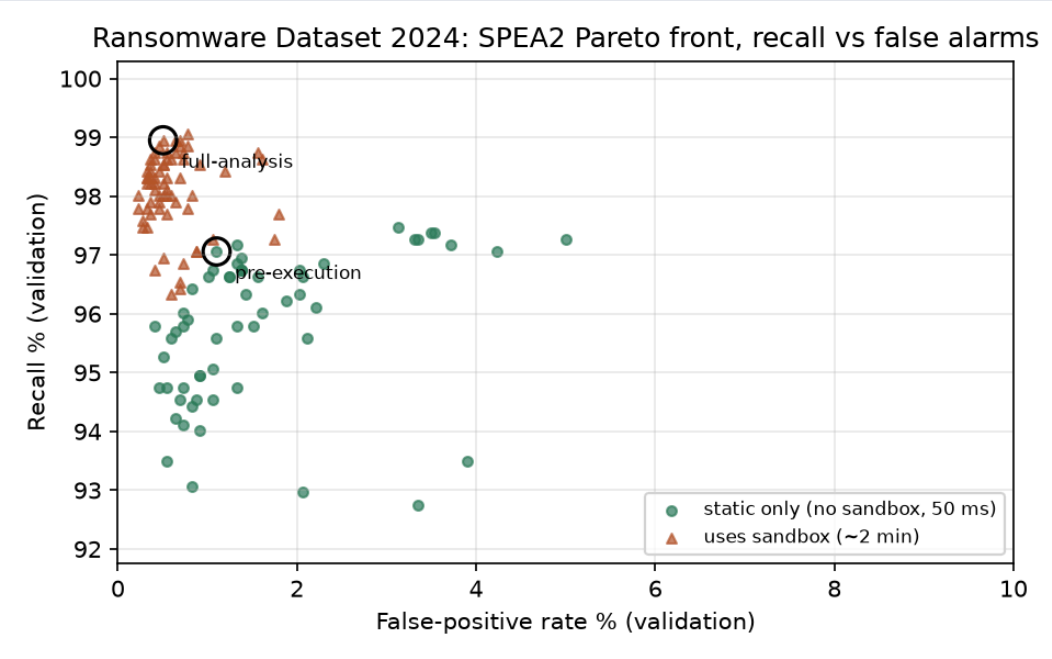


Figure 1. RD2024, SPEA2 pooled Pareto front (validation data). Green: static-only detectors (50 ms). Orange: detectors that need the sandbox. Circles mark the two chosen deployment points. Detectors above 10% FPR (the flag-everything tail) are off the chart.

7.1 Weighted-sum PSO

PSO ran four weightings of the three objectives from the same sparse start.

- **Weights did not steer it.** On RD2024 all four weightings, from "cost first" to "false alarms first", ended at almost the same static-only detector (56–71 features). There was never a sandbox option to choose from, whereas one SPEA2 run returns about 100 trade-offs.
- **It cannot stay sparse.** Binary PSO redraws every bit each iteration with probability sigmoid(velocity), which keeps about half the features switched on. On MLRan, where 260 dynamic bits must all be off, the swarm never saw a good static detector again. The "balanced" and "cost" weightings returned a 3-feature detector that flags 95% of benign software. It is cheap, so the weighted score preferred it.
- **Where it worked, it used far more features:** 68 against SPEA2's 7 on RD2024, at similar accuracy.

7.2 What the selected features are

RD2024's 7-feature pre-execution detector uses `ImageBase`, `SizeofStackCommit`, three properties of the `.rdata` section (size, address, raw size), and two section flags. These are layout properties of how the file was built: ransomware builders and packers leave a recognisable fingerprint. `ImageBase` appears in 52% of all SPEA2 Pareto solutions. On

MLRan the most-selected static strings include `getlogicaldrives` (enumerating drives to encrypt), `removedirectorya` and `getexitcodeprocess`: plausible ransomware behaviour, visible in the file before it runs.

7.3 An honest caveat: small validation sets over-fit

MLRan's validation split holds only about 500 benign programs, so one false alarm is 0.2%. Choosing the best of 265 Pareto solutions on such a small set rewards luck. SPEA2's full-analysis pick scored **0.2% FPR in validation but 5.1% on the test set**. NSGA-II's pick happened to generalise better. On RD2024, with about 2,000 benign validation programs, validation and test agree closely (SPEA2 pre-execution: 1.10% → 1.33%).

7.4 UGRansome

UGRansome is distributed only through Kaggle, which needs an account login, so it could not be downloaded automatically. The loader (zero-day "A" vs known-signature "S" traffic, text attributes frequency-encoded), audit, search and an **exhaustive search over every feature subset** are implemented and tested. With about 13 features (about 8,000 subsets), the exhaustive search gives the *true* Pareto front. This lets us measure how much of the optimum SPEA2 and NSGA-II actually recover (share of the true front, and inverted generational distance). Results will be added once the file is placed in `data/ugransome/`.

8. Conclusion

Yes, ransomware can be caught before it runs, at least in RD2024. SPEA2 found a detector that reads **7 PE-header fields in about 50 ms**, never executes the file, and reaches **96.12% recall at 1.33% false alarms** on unseen test data. Both targets are met. Compared with the full 148-feature sandbox pipeline (98.74% / 0.64%), skipping the sandbox costs 2.6 points of recall and 0.7 points of false alarms, and makes detection about 2,400 times faster.

- **On MLRan, static analysis falls short.** The best static-only detector (SPEA2, 73 strings) reaches 93.33% recall at 4.90% FPR; on the 655 genuinely unseen samples, 94.85% at 8.74%. That is 2.4 points of recall better than using all 223 strings, but below target. Reaching ≥ 95% requires the sandbox. NSGA-II's 158-feature detector (98.06% / 3.33%) beat the 483-feature baseline with a third of the features.
- **Best algorithm.** SPEA2 and NSGA-II are statistically tied in search quality (Mann–Whitney p = 0.68 on MLRan, 1.00 on RD2024), as in the phishing study. SPEA2 produced the best, and the most compact, pre-execution detector on both datasets; NSGA-II produced the best full-analysis detector on MLRan. Both clearly beat weighted-sum PSO, which returned one point per run and on MLRan collapsed to a useless detector. The filter and PCA either needed the sandbox or all inputs.
- **Method lesson.** At 483 features, the standard random start makes sandbox-free detectors unreachable (0 found in 6 runs). A sparse start, which uses no knowledge of the tiers, found 47–91 per run. Large feature spaces need this.

Limitations: subsets are tuned for a Random Forest. MLRan's small validation set lets the final choice over-fit (7.3). Static features can be evaded by packing, so a pre-execution filter complements a sandbox rather than replacing it. UGRansome results await the download.